INF321 - CONCEPTION ET ANALYSE DES ALGORITHMES

INVESTIGATION ALGORITHMIQUE ET EXPÉRIMENTALE DES PROBLÈMES NP-COMPLETS

Présenté par :

NGUEYA PAMBI AUDREY VIVIANE	23U2997	SAT
DJUOMBESSY DJUOMBESSY FORTUNE ANTOINETTE	23U2814	3-SAT
TCHUENCHE KAMDEM SAMUEL YEDIDYA	23U2987	CLIQUE
MAKUNE KOUGOUM BIJOEL	23V2141	VERTEX-COVER
DJOUTA WAMEGNE LUCIANA	23V2283	HAMPATH
FOKOU KENGNI SHERYLE ALANE	23V2378	UHAMPATH
KAMDOM NGUETCHESSI MERVEILLE PRISCA	23V2060	SUBSET SUM

Sous la direction de :
DR. MESSI NGUELE THOMAS

Année Académique 2025-2026

SOMMAIRE

INTRODUCTION GÉNÉRALE 1

- La distinction fondamentale entre les classes **P** et **NP**
- La problématique du défi mathématique **P = NP**
- Interconnectivité et réduction polynomiale des problèmes **NP-complets**

I. LE PROBLÈME SAT (SATISFIABILITÉ GÉNÉRALE) 3

- 1.1. Définitions : Variables, Littéraux et Forme Normale Conjonctive (FNC)
- 1.2. Démonstration de la NP-complétude : Théorème de Cook-Levin
- 1.3. Implémentation en C : Algorithme de Backtracking et complexité

II. LE PROBLÈME 3-SAT (VERSION RESTREINTE) 7

- 2.1. Définitions : Structure des clauses à trois littéraux (3-CNF)
- 2.2. Démonstration : Réduction polynomiale de SAT vers 3-SAT
- 2.3. Implémentation en C : Adaptation de l'algorithme de résolution

III. LE PROBLÈME CLIQUE (THÉORIE DES GRAPHES) 12

- 3.1. Définitions : Sous-graphes complets et matrice d'adjacence
- 3.2. Démonstration : Réduction de 3-SAT vers CLIQUE
- 3.3. Implémentation en C : Recherche récursive de sous-ensembles connectés

IV. LE PROBLÈME VERTEX-COVER (COUVERTURE DE SOMMETS) 18

- 4.1. Définitions : Couverture des arêtes par un ensemble minimal de sommets
- 4.2. Démonstration : Preuve de NP-complétude par réduction depuis CLIQUE
- 4.3. Implémentation en C : Algorithme de vérification et d'exploration

V. LE PROBLÈME HAMPATH (CHEMIN HAMILTONIEN ORIENTÉ) 24

- 5.1. Définitions : Graphes orientés et parcours exhaustif simple
- 5.2. Démonstration : Complexité du cycle et du chemin hamiltonien
- 5.3. Implémentation en C : Gestion des directions et des visites uniques

VI. LE PROBLÈME UHAMPATH (CHEMIN HAMILTONIEN NON-ORIENTÉ) 29

- 6.1. Définitions : Symétrie des arêtes et connectivité

- 6.2. Démonstration : Réduction de HAMPATH vers UHAMPATH
- 6.3. Implémentation en C : Algorithme de backtracking symétrique

VII. LE PROBLÈME SUBSET-SUM (SOMME DE SOUS-ENSEMBLE) 34

- 7.1. Définitions : Ensembles d'entiers et valeur cible (T)
- 7.2. Démonstration : Lien avec le problème du sac à dos (Knapsack)
- 7.3. Implémentation en C : Exploration binaire et élagage des sommes

ANALYSE COMPARATIVE ET BILAN 40

- Tableau comparatif des sept problèmes (Certificats, Réductions, Domaines)
- Analyse des résultats : Temps réel et explosion combinatoire sur machine
- La réalité de l'intraitabilité face à l'augmentation de la taille des entrées

CONCLUSION 44

- Synthèse sur l'importance des heuristiques et de l'optimisation
- Réflexion finale sur l'avenir du défi P versus NP

INTRODUCTION

L'informatique théorique repose sur la distinction entre les problèmes résolubles efficacement (**P**) et ceux dont la solution est seulement vérifiable en temps polynomial (**NP**). **D'une part**, la question fondamentale de savoir si **P=NP** demeure l'un des plus grands défis mathématiques actuels, car elle interroge la possibilité d'automatiser la recherche de solutions complexes.

C'est dans ce cadre que s'inscrit la classe des problèmes **NP-complets**. Ces derniers représentent les problèmes les plus difficiles de la classe NP ; ils possèdent la propriété remarquable d'être interconnectés. **Par conséquent**, si un algorithme polynomial était découvert pour l'un d'eux, tous les problèmes NP seraient résolus instantanément par simple **réduction polynomiale**.

Pour illustrer cette théorie, ce Travail Pratique analyse sept problèmes piliers de la complexité :**SAT** et **3-SAT** (Logique), **CLIQUE** et **Vertex-Cover** (Graphes), **HAMPATH** et **UHAMPATH** (Chemins), **SUBSET-SUM** (Arithmétique)

Enfin, au-delà des démonstrations formelles, ce rapport confronte la théorie à la pratique(réalité machine). En l'absence de solutions polynomiales connues, nous utiliserons le **langage C** pour implémenter ces algorithmes. Cette approche permettra de mettre en évidence l'**explosion exponentielle** du temps de calcul : dès que la taille de l'entrée augmente, le temps de résolution sature la machine, prouvant ainsi l'intraitabilité réelle de ces problèmes.

I. PROBLÈME SAT : SATISFIABILITÉ

1. Définition du Problème SAT

Le problème SAT (Boolean Satisfiability Problem) est le problème fondamental de la logique booléenne. Il se définit comme suit :

- **Entrées : Variables Booléennes** : Un ensemble de variables $\{x_1, x_2, \ldots, x_n\}$ pouvant prendre les valeurs $\{0, 1\}$ (Faux/Vrai).
  - **Littéral** : Une variable (x_i) ou sa négation $(\neg x_i)$.
  - **Clause** : Une disjonction (OU logique) de un ou plusieurs littéraux (ex : $C = x_1 \vee \neg x_2 \vee x_3$).
  - **Formule en Forme Normale Conjonctive (FNC)** : Une conjonction (ET logique) de plusieurs clauses $F = C_1 \wedge C_2 \wedge \cdots \wedge C_k$.

Problème : Étant donnée une formule F en FNC, existe-t-il une affectation de vérité des variables telle que F soit évaluée à **Vrai** ?

2. Démonstration de la NP-Complétude

Pour prouver qu'un problème est NP-complet, nous devons démontrer son appartenance à la classe NP et sa NP-difficulté.

A. SAT appartient à NP (SAT $\in$ NP)

- **Certificat** : Une affectation de vérité pour les n variables de la formule.
- **Algorithme de Vérification** : Pour une affectation donnée, l'algorithme parcourt chaque clause. Une clause est satisfaite si au moins l'un de ses littéraux est vrai. La formule est satisfaite si toutes ses clauses sont vraies.
- **Complexité** : Cette vérification s'effectue en temps polynomial $O(L)$, où L est la longueur totale de la formule (nombre de littéraux). Puisqu'un certificat peut être vérifié en temps polynomial, **SAT $\in$ NP**.

B. SAT est NP-difficile (Théorème de Cook-Levin)

Le théorème de Cook-Levin établit que SAT est le point d'ancrage de la classe NP-complet. Tout problème $P' \in$ NP se réduit polynomialement à SAT :

$$P' \leq_p \text{SAT}$$

Principe de la Réduction : Toute instance d'un problème dans NP est décidée par une Machine de Turing Non-Déterministe (MTND) en temps polynomial $T(n)$.

Simulation par Formule : On construit une formule booléenne ϕ qui encode le fonctionnement de cette MTND. Cette formule utilise des variables pour représenter :

- L'état de la machine à chaque instant $t \leq T(n)$.
- Le contenu des cases de la bande à chaque instant t.
- La position de la tête de lecture à chaque instant t.

Contraintes : Des clauses imposent que la machine suit ses règles de transition, qu'elle commence avec l'entrée sur la bande et qu'elle termine dans un état d'acceptation.

Conclusion de la réduction : La formule ϕ est satisfiable si et seulement si la MTND accepte l'entrée. La construction de ϕ étant polynomiale, SAT est NP-difficile.

Puisque SAT est à la fois dans NP et NP-difficile, **il est NP-complet**.
C'est le premier problème à avoir été identifié comme tel, servant de base à toutes les autres preuves de NP-complétude par réduction.

3. Implémentation du programme en C pour le NP-Complet "SAT"

Pour SAT, comme c'est un problème NP-complet, nous allons utiliser un algorithme de **Backtracking** (exploration par essai-erreur). C'est l'approche standard pour montrer comment le temps de calcul grimpe exponentiellement.

Algorithme de référence : Le backtracking est la preuve par excellence de la complexité exponentielle. Pour n=5, c'est instantané. Pour n=30, l'utilisateur verra le programme « réfléchir », illustrant physiquement la difficulté du problème NP. C'est une étape cruciale.

```
#include "np_complet.h"

typedef struct {
 int* literaux;
 int nbLiteraux;
} Clause;

bool clauseSatisfaite(int* lits, int nb, int* assign) {
 for (int j = 0; j < nb; j++) {
 int lit = lits[j];
 int var = abs(lit) - 1;
 int val = assign[var];
 if ((lit > 0 && val == 1) || (lit < 0 && val == 0)) return true;
 }
 return false;
}

bool resoudreSAT(Clause* formules, int m, int n, int cur, int* assign, int* solution) {
 if (cur == n) {
 for (int i = 0; i < m; i++) {
 if (!clauseSatisfaite(formules[i].literaux, formules[i].nbLiteraux, assign)) return false;
 }
 memcpy(solution, assign, n * sizeof(int));
 return true;
 }
```

```
    for (int v = 0; v <= 1; v++) {
        assign[cur] = v;
        if (resoudreSAT(formules, m, n, cur + 1, assign, solution)) return true;
    }
    return false;
}

void executionSAT() {
    char choix;
    afficherEntete("MODULE SAT (général)");

    printf("Générer les clauses aléatoirement ? (o/n) : ");
    scanf(" %c", &choix);

    int n, m;
    printf("Nombre de variables : ");
    scanf("%d", &n);
    printf("Nombre de clauses    : ");
    scanf("%d", &m);

    Clause* formules = malloc(m * sizeof(Clause));

    if (choix == 'o' || choix == 'O') {
        srand(time(NULL));
        printf("Génération aléatoire des %d clauses pour %d variables...\n", m, n);

        for (int i = 0; i < m; i++) {
            int nbLit = 1 + rand() % 6;  // 1 à 6 littéraux
            formules[i].nbLiteraux = nbLit;
            formules[i].literaux = malloc(nbLit * sizeof(int));

            for (int j = 0; j < nbLit; j++) {
                int var = 1 + rand() % n;
                int signe = (rand() % 2) ? 1 : -1;
                formules[i].literaux[j] = signe * var;
            }
        }
    } else {
        for (int i = 0; i < m; i++) {
            printf("Clause %d - Nombre de littéraux : ", i + 1);
            scanf("%d", &formules[i].nbLiteraux);
            formules[i].literaux = malloc(formules[i].nbLiteraux * sizeof(int));
            printf("Entrez les littéraux (ex: 1 -2 3) : ");
            for (int j = 0; j < formules[i].nbLiteraux; j++) {
                scanf("%d", &formules[i].literaux[j]);
            }
        }
    }

    int* assignation = calloc(n, sizeof(int));
```

```
    int* solution = malloc(n * sizeof(int));

    printf("\nRecherche d'assignation satisfaisante en cours...\n");

    clock_t debut = clock();
    bool satisfiable = resoudreSAT(formules, m, n, 0, assignation, solution);
    clock_t fin = clock();
    double temps = (double)(fin - debut) / CLOCKS_PER_SEC;

    if (satisfiable) {
        printf("\nRESULTAT : La formule est SATISFIABLE.\n");
        printf("Raison : Une assignation valide a été trouvée :\n   ");
        for (int i = 0; i < n; i++) {
            printf("x%d=%d ", i + 1, solution[i]);
        }
        printf("\n");
    } else {
        printf("\nRESULTAT : La formule est NON SATISFIABLE (UNSAT).\n");
        printf("Raison : Aucune assignation ne satisfait toutes les clauses.\n");
    }

    printf("TEMPS D'EXÉCUTION : %.6f secondes\n", temps);
    estimerEnergieSimple(temps);

    // Nettoyage
    for (int i = 0; i < m; i++) free(formules[i].literaux);
    free(formules);
    free(assignation);
    free(solution);
}
```

4. Execution

```
============================================
      PLATEFORME D'EXPERIMENTATION NP-COMPLET
============================================
 1. SAT (général)
 2. 3-SAT
 3. CLIQUE
 4. VERTEX-COVER
 5. HAMPATH (orienté)
 6. UHAMPATH (non orienté)
 7. SUBSET-SUM
 0. Quitter
 Choix : 1

╔═══════════════════════════════════════════╗
                MODULE SAT (général)
╚═══════════════════════════════════════════╝

Générer les clauses aléatoirement ? (o/n) : o
Nombre de variables : 18
Nombre de clauses    : 40
Génération aléatoire des 40 clauses pour 18 variables...

Recherche d'assignation satisfaisante en cours...

RESULTAT : La formule est SATISFIABLE.
Raison : Une assignation valide a été trouvée :
   x1=1 x2=0 x3=0 x4=0 x5=1 x6=1 x7=1 x8=1 x9=0 x10=0 x11=0 x12=0 x13=1 x14=0 x15=0 x16=0 x17=0 x18=0
TEMPS D'EXÉCUTION : 0.019852 secondes
Énergie estimée ≈ 0.496 Joules

Appuyez sur Entrée pour retourner au menu...
```

II. PROBLÈME 3-SAT

1. Définition du problème 3-SAT

Une formule est en **Forme Normale Conjonctive de taille 3 (3-CNF)** si elle est une conjonction de clauses où chaque clause est une disjonction d'**au plus trois littéraux**.

Le problème 3-SAT se définit comme suit :

- **Entrée** : Une formule ϕ en 3-CNF.
- **Question** : Existe-t-il une assignation des variables qui rende ϕ vraie ?

2. Démonstration de la NP-Complétude

Objectif : Montrer que 3-SAT est NP-complet. Pour cela, nous devons prouver que :

- $3\text{-SAT} \in \text{NP}$
- 3-SAT est NP-difficile, c'est-à-dire qu'il existe une réduction polynomiale depuis SAT (déjà prouvé NP-complet par le théorème de Cook-Levin) :
  $\text{SAT} \leq_p 3\text{-SAT}$

A. Appartenance à NP ($3\text{-SAT} \in \text{NP}$)

Preuve : Une assignation des variables constitue un certificat. Pour une formule ϕ en 3-CNF comportant n variables et m clauses, vérifier si une assignation satisfait ϕ consiste à évaluer chaque clause (au plus 3 littéraux). Cette vérification s'effectue en temps $O(m)$, ce qui est polynomial par rapport à la taille de l'entrée.

$\Rightarrow 3\text{-SAT} \in \text{NP}$

B. Réduction : Transformation d'une formule CNF quelconque en 3-CNF

On part d'une instance de SAT (formule F en CNF dont les clauses ont une taille quelconque). On construit en temps polynomial une formule F' en 3-CNF telle que :

$$F \text{ est satisfiable} \Leftrightarrow F' \text{ est satisfiable}$$

B.1 Cas des clauses courtes (taille ≤ 3)

- Clause à 1 littéral : $(x) \mapsto (x \vee x \vee x)$
- Clause à 2 littéraux : $(x \vee y) \mapsto (x \vee y \vee y)$
- Clause à 3 littéraux : inchangée

B.2 Transformation d'une clause longue ($k \geq 4$)

Soit $C = (x_1 \vee x_2 \vee x_3 \vee \ldots \vee x_k)$.
On introduit $k-3$ nouvelles variables auxiliaires $y_1, y_2, \ldots, y_{k-3}$.

On remplace C par la conjonction des $k-2$ clauses suivantes :

$$(x_1 \vee x_2 \vee y_1)$$
$$(\neg y_1 \vee x_3 \vee y_2)$$
$$(\neg y_2 \vee x_4 \vee y_3)$$
$$\vdots$$
$$(\neg y_{k-4} \vee x_{k-2} \vee y_{k-3})$$
$$(\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

Exemple concret (k=5) :

$$(x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5) \mapsto (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_4 \vee x_5)$$

B.3 Intuition de la construction

Les variables y_i servent à « transmettre » l'obligation de satisfaire au moins un des littéraux originaux. Si aucun des x_1, x_2 n'est vrai, alors y_1 doit être vrai. Si y_1 est vrai, alors x_3 ou y_2 doit être vrai, et ainsi de suite. Finalement, au moins un des x_j doit être vrai pour que l'ensemble soit satisfait.

B.4 Preuve d'équisatisfiabilité

- $(\Rightarrow)$ Si C est satisfaite par une assignation A (au moins un x_j vrai), on peut étendre A aux y_i (souvent en les fixant à faux).

- $(\Leftarrow)$ Si toutes les clauses de la transformation sont satisfaites et que (par l'absurde) tous les x_j sont faux, alors la première clause force y_1 vrai, la seconde force y_2 vrai, … jusqu'à la dernière clause qui force x_{k-1} ou x_k vrai → contradiction.

B.5 Cas particuliers

- Clause vide ($\perp$) : On produit une formule 3-CNF insatisfiable, par exemple : $(a \vee a \vee a) \wedge (\neg a \vee \neg a \vee \neg a)$

B.6 Complexité de la réduction

- Nombre de nouvelles variables : $O(\text{taille de } F)$

- Nombre total de clauses dans F' : $O(\text{taille de } F)$

- Temps de construction : linéaire (parcours simple des clauses)

C. Validité globale de la réduction

Pour toute formule CNF F, on construit F' comme décrit :

- Si F est satisfiable → l'assignation peut être étendue aux $y_i \rightarrow F'$ satisfiable
- Si F' est satisfiable → la restriction aux variables originales satisfait F

⇒ Équisatisfiabilité préservée : F satisfiable $\Leftrightarrow$ F' satisfiable

D. Conclusion

Nous avons montré que :

- $3\text{-SAT} \in \text{NP}$
- $\text{SAT} \leq_p 3\text{-SAT}$

Puisque SAT est NP-complet, et que la réduction est polynomiale, par transitivité tout problème de NP se réduit à 3-SAT.

Donc 3-SAT est NP-complet.

3. Implémentation du programme en C pour le NP-Complet "3-SAT"

Pour le code de **3-SAT**, nous construisons un module similaire à celui de SAT, mais avec une structure de données plus rigide (maximum 3 littéraux par clause). Comme 3-SAT est un cas particulier de SAT, l'algorithme reste le même (backtracking), mais l'interface de saisie force l'utilisateur à respecter la contrainte des 3 littéraux pour bien illustrer le problème spécifique du TP.

```
#include "np_complet.h"

typedef struct {
 int L1, L2, L3;
 int nbActif;
} Clause3;

bool estClauseSatisfaite(Clause3 c, int* assignation) {
 int lits[3] = {c.L1, c.L2, c.L3};
 for (int i = 0; i < c.nbActif; i++) {
 int varIdx = abs(lits[i]) - 1;
 int val = assignation[varIdx];
 if ((lits[i] > 0 && val == 1) || (lits[i] < 0 && val == 0)) {
 return true;
 }
 }
 return false;
}

bool resoudre3SAT(Clause3* formules, int m, int n, int cur, int* assign, int* solution) {
 if (cur == n) {
 for (int i = 0; i < m; i++) {
 if (!estClauseSatisfaite(formules[i], assign)) return false;
 }
```

```
        memcpy(solution, assign, n * sizeof(int));
        return true;
    }

    for (int v = 0; v <= 1; v++) {
        assign[cur] = v;
        if (resoudre3SAT(formules, m, n, cur + 1, assign, solution)) return true;
    }
    return false;
}

void execution3SAT() {
    char choix;
    afficherEntete("MODULE 3-SAT (strict 3 littéraux)");

    printf("Générer les clauses aléatoirement ? (o/n) : ");
    scanf(" %c", &choix);

    int n, m;
    printf("Nombre de variables : ");
    scanf("%d", &n);
    printf("Nombre de clauses    : ");
    scanf("%d", &m);

    Clause3* formules = malloc(m * sizeof(Clause3));

    if (choix == 'o' || choix == 'O') {
        srand(time(NULL));
        printf("Génération aléatoire des %d clauses (3 littéraux)...\n", m);
        for (int i = 0; i < m; i++) {
            int v1 = 1 + rand() % n;
            int v2 = 1 + rand() % n;
            int v3 = 1 + rand() % n;
            formules[i].L1 = (rand() % 2 ? 1 : -1) * v1;
            formules[i].L2 = (rand() % 2 ? 1 : -1) * v2;
            formules[i].L3 = (rand() % 2 ? 1 : -1) * v3;
            formules[i].nbActif = 3;
        }
    } else {
        for (int i = 0; i < m; i++) {
            printf("Clause %d (L1 L2 L3, 0 si absent) : ", i + 1);
            int l1, l2, l3;
            scanf("%d %d %d", &l1, &l2, &l3);
            formules[i].L1 = l1;
            formules[i].L2 = l2;
            formules[i].L3 = l3;
            formules[i].nbActif = (l3 != 0) ? 3 : (l2 != 0 ? 2 : 1);
        }
    }
```

```
    int* assignation = calloc(n, sizeof(int));
    int* solution = malloc(n * sizeof(int));

    printf("\nRecherche en cours...\n");

    clock_t start = clock();
    bool possible = resoudre3SAT(formules, m, n, 0, assignation, solution);
    clock_t end = clock();

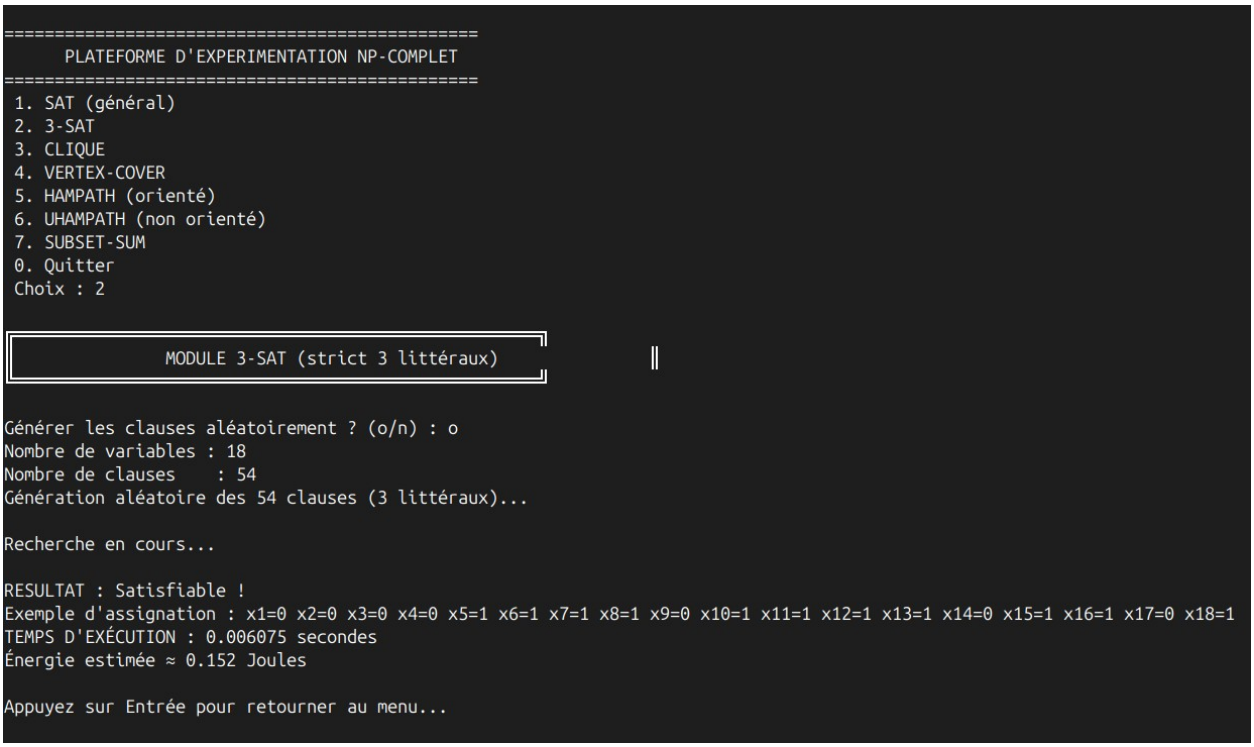
    double temps = (double)(end - start) / CLOCKS_PER_SEC;

    if (possible) {
        printf("\nRESULTAT : Satisfiable !\n");
        printf("Exemple d'assignation : ");
        for (int i = 0; i < n; i++) printf("x%d=%d ", i + 1, solution[i]);
        printf("\n");
    } else {
        printf("\nRESULTAT : Insatisfiable.\n");
    }

    printf("TEMPS D'EXÉCUTION : %.6f secondes\n", temps);
    estimerEnergieSimple(temps);

    free(formules);
    free(assignation);
    free(solution);
}
```

4. Execution


PLATEFORME D'EXPERIMENTATION NP-COMPLET
1. SAT (général)
2. 3-SAT
3. CLIQUE
4. VERTEX-COVER
5. HAMPATH (orienté)
6. UHAMPATH (non orienté)
7. SUBSET-SUM
0. Quitter
Choix : 2
MODULE 3-SAT (strict 3 littéraux)
Générer les clauses aléatoirement ? (o/n) : o
Nombre de variables : 18
Nombre de clauses : 54
Génération aléatoire des 54 clauses (3 littéraux)...
Recherche en cours...
RESULTAT : Satisfiable !
Exemple d'assignation : x1=0 x2=0 x3=0 x4=0 x5=1 x6=1 x7=1 x8=1 x9=0 x10=1 x11=1 x12=1 x13=1 x14=0 x15=1 x16=1 x17=0 x18=1
TEMPS D'EXÉCUTION : 0.006075 secondes
Énergie estimée ≈ 0.152 Joules
Appuyez sur Entrée pour retourner au menu...

III. LE PROBLÈME CLIQUE

1. Définition du Problème CLIQUE

1.1 Définitions Préliminaires

Définition 1.1 (Graphe non orienté)
Un graphe non orienté est un couple $G=(V,E)$ où :
- V est un ensemble fini de **sommets** (vertices),
- $E \subseteq \{\{u,v\} \vee u,v \in V, u \neq v\}$ est un ensemble d'**arêtes** (edges).

Définition 1.2 (Clique)
Soit $G=(V,E)$ un graphe non orienté. Une **clique** dans G est un sous-ensemble de sommets $C \subseteq V$ tel que pour toute paire de sommets distincts $u,v \in C$, l'arête $\{u,v\}$ appartient à E.

1.2 Définition Formelle du Problème

- **Instance** : Un graphe non orienté $G=(V,E)$ et un entier positif k.
- **Question** : Le graphe G contient-il une **clique de taille** k ?
- **Langage** :
  CLIQUE$=\{\langle G,k\rangle \vee G \text{ contient une clique de taille } k\}$.

2. Démonstration de la NP-Complétude

2.1 Préliminaires Théoriques

Définition 2.1 (NP-Complet)
Un langage B est **NP-complet** s'il satisfait deux conditions :
- $B \in$ NP,
- Pour tout $A \in$ NP, $A \leq_p B$.

Définition 2.2 (Réductibilité polynomiale)
Un langage A est réductible en temps polynomial à un langage B, noté $A \leq_p B$, s'il existe une fonction calculable en temps polynomial $f : \Sigma^{\square} \to \Sigma^{\square}$ telle que pour tout $w \in \Sigma^{\square}$:

$$w \in A \Leftrightarrow f(w) \in B$$

2.2 Première Condition : CLIQUE ∈ NP

Preuve :
Soit $\langle G,k\rangle$ une instance de CLIQUE.
Un certificat c est un sous-ensemble $C \subseteq V$ avec $|C|=k$.

L'algorithme de vérification procède comme suit :

1. Vérifier que $|C|=k \to$ Temps : $O(1)$

2. Pour chaque paire de sommets distincts $u, v \in C$, vérifier que $\{u, v\} \in E$
   Nombre de paires : $\binom{k}{2} = \frac{k(k-1)}{2}$
   Temps total pour la vérification : $O(k^2)$

Le temps de vérification est polynomial en la taille de l'entrée ⇒ **CLIQUE ∈ NP**.

2.3 Deuxième Condition : CLIQUE est NP-Difficile

Nous démontrons que **3-SAT ≤_p CLIQUE**.

2.3.1 Rappel de 3-SAT

- **Instance** : Une formule booléenne ϕ en forme normale conjonctive avec **exactement 3 littéraux par clause** :
  $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$
  où chaque $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$.

2.3.2 Construction de la Réduction

On construit $f(\langle \phi \rangle) = \langle G, k \rangle$ comme suit :

- **Ensemble de sommets** V :
  $V = \{v_{ij} \vee 1 \le i \le m, 1 \le j \le 3\}$
  Chaque sommet v_{ij} représente le j-ème littéral de la i-ème clause.
- **Ensemble d'arêtes** E :
  Pour deux sommets v_{ij} et v_{pq} avec $i \neq p$, on ajoute l'arête $\{v_{ij}, v_{pq}\}$ **sauf si** :
  - $i = p$ (même clause), **ou**
  - $l_{ij} = \neg l_{pq}$ (littéraux contradictoires).
- **Paramètre** : $k = m$

2.3.3 Complexité de la construction

- Nombre de sommets : $3m$
- Nombre de paires de sommets à considérer : $O((3m)^2) = O(m^2)$
- Vérification par paire : $O(1)$
  → Temps total de construction : $O(m^2)$ → **polynomial**

2.3.4 Preuve d'équivalence

$\phi \in$ 3-SAT $\Leftrightarrow \langle G, m \rangle \in$ CLIQUE

Sens $(\Rightarrow)$:
Supposons ϕ satisfiable. Il existe une affectation τ qui rend ϕ vraie.
Dans chaque clause C_i, au moins un littéral l_{ij} est vrai sous τ.

On choisit **un** tel littéral par clause → on obtient un ensemble C de m sommets.
C est une clique car :

- Pas deux sommets de la même clause (un seul par clause)
- Pas de littéraux contradictoires (ils sont tous vrais sous τ)
  → Toutes les arêtes nécessaires existent.

Sens $(\Leftarrow)$:
Supposons qu'il existe une clique C de taille m dans G.
Alors :

- C contient **exactement un sommet par clause** (car pas d'arêtes intra-clause)
- C ne contient **pas de littéraux contradictoires** (car pas d'arêtes entre eux)

On définit l'affectation τ en mettant **VRAI** tous les littéraux correspondant aux sommets de C.
Chaque clause possède donc au moins un littéral vrai → ϕ est satisfaite.

2.4 Conclusion de la NP-Difficulté

Nous avons montré une réduction polynomiale **3-SAT ≤_p CLIQUE**.**
Puisque 3-SAT est NP-complet et que la réductibilité polynomiale est transitive, CLIQUE est NP-difficile

Ayant démontré que :

- CLIQUE ∈ NP
- CLIQUE est NP-difficile

nous concluons que CLIQUE est NP-complet.

3. Implémentation du programme en C pour le NP-Complet "CLIQUE"

Pour le code, le problème est différent de SAT. Ici, nous manipulons généralement une **matrice d'adjacence** pour représenter le graphe.

```c
#include "np_complet.h"

static int graphe[100][100];
static int sommets_clique[100];
static int solution_clique[100];
static int n_sommets, k_cible;

bool estUneClique(int taille) {
    for (int i = 0; i < taille; i++)
        for (int j = i + 1; j < taille; j++)
            if (graphe[sommets_clique[i]][sommets_clique[j]] == 0) return false;
    return true;
}
```

```
bool trouverClique(int debut, int taille_actuelle) {
    if (taille_actuelle == k_cible) {
        if (estUneClique(k_cible)) {
            memcpy(solution_clique, sommets_clique, k_cible * sizeof(int));
            return true;
        }
        return false;
    }
    for (int i = debut; i < n_sommets; i++) {
        sommets_clique[taille_actuelle] = i;
        if (trouverClique(i + 1, taille_actuelle + 1)) return true;
    }
    return false;
}

void executionClique() {
    char choix;
    afficherEntete("MODULE CLIQUE");

    printf("Générer aléatoirement ? (o/n) : "); scanf(" %c", &choix);

    printf("Nombre de sommets : "); scanf("%d", &n_sommets);
    printf("Taille clique (k) : "); scanf("%d", &k_cible);

    if (k_cible > n_sommets) { printf("Erreur : k > n\n"); return; }

    if (choix == 'o' || choix == 'O') {
        srand(time(NULL));
        float p = 0.35f + (rand()/(float)RAND_MAX)*0.45f;
        printf("Densité ≈ %.2f\n", p);
        for (int i = 0; i < n_sommets; i++)
            for (int j = 0; j < n_sommets; j++)
                graphe[i][j] = (i==j)?0:((rand()%1000) < p*1000);
    } else {
        printf("Matrice :\n");
        for (int i = 0; i < n_sommets; i++)
            for (int j = 0; j < n_sommets; j++)
                scanf("%d", &graphe[i][j]);
    }

    afficherMatriceAdjacence(graphe, n_sommets, "Matrice d'adjacence");
    genererDotFile(graphe, n_sommets, "graph.dot", false);

    printf("\nRecherche clique de taille %d...\n", k_cible);

    clock_t d = clock();
    bool ok = trouverClique(0, 0);
    clock_t f = clock();
    double t = (double)(f - d) / CLOCKS_PER_SEC;
```

```
    if (ok) {
        printf("\nCLIQUE TROUVÉE de taille %d\n{ ", k_cible);
        for (int i = 0; i < k_cible; i++) printf("%d%s", solution_clique[i], i<k_cible-1?", ":" }\n");
    } else {
        printf("\nPas de clique de taille %d\n", k_cible);
    }

    printf("Temps : %.6f s\n", t);
    estimerEnergieSimple(t);

    proposerEtAfficherGraphe("graph.dot", "graphe.png");

}
```

Note sur le code : Ce programme illustre bien la complexité. Pour un graphe de 20 sommets, si nous cherchons une clique de taille 10, le nombre de combinaisons est de $\binom{20}{10} = 184\,756$. Imaginons alors pour une clique de taille 30, 40…

4. Execution

```
                MODULE CLIQUE                ‖

Générer aléatoirement ? (o/n) : o
Nombre de sommets : 18
Taille clique (k) : 6
Densité ≈ 0.76

Matrice d'adjacence (18 sommets) :
     0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
 0 |  0   1   1   1   1   0   1   0   1   0   1   1   1   1   1   1   0   1
 1 |  1   0   0   0   0   0   1   1   1   1   1   1   0   1   0   1   1   1
 2 |  1   0   0   1   0   1   0   1   1   0   1   1   1   1   0   1   1   1
 3 |  0   1   1   0   0   0   1   1   1   0   0   1   1   1   1   1   1   1
 4 |  1   1   0   1   0   1   1   1   1   1   1   1   1   1   1   1   0   1
 5 |  1   0   1   1   0   0   1   1   1   0   1   0   0   0   1   1   1   1
 6 |  0   0   1   1   1   1   0   0   1   1   1   0   1   1   1   1   1   1
 7 |  1   1   0   1   1   1   1   0   1   1   1   0   0   1   1   0   1   0
 8 |  1   0   1   1   1   0   0   1   0   1   1   0   1   1   1   0   1   1
 9 |  1   1   1   1   1   1   1   1   1   0   1   0   0   1   1   1   1   1
10 |  0   1   1   1   0   0   1   0   1   0   0   1   1   1   1   1   1   1
11 |  1   1   1   1   1   1   1   1   1   1   1   0   0   1   1   1   1   1
12 |  1   1   1   1   1   1   0   1   0   1   0   1   0   1   0   1   1   1
13 |  1   1   0   0   1   1   1   0   1   0   1   1   1   0   1   0   1   1
14 |  0   1   0   1   1   0   1   0   1   1   0   1   1   1   0   1   0   1
15 |  1   1   1   0   1   1   0   1   1   1   1   1   1   1   1   0   0   1
16 |  1   1   1   1   1   1   1   1   1   0   0   1   1   1   1   1   0   1
17 |  1   1   1   1   1   1   0   1   0   1   1   1   1   0   1   1   1   0

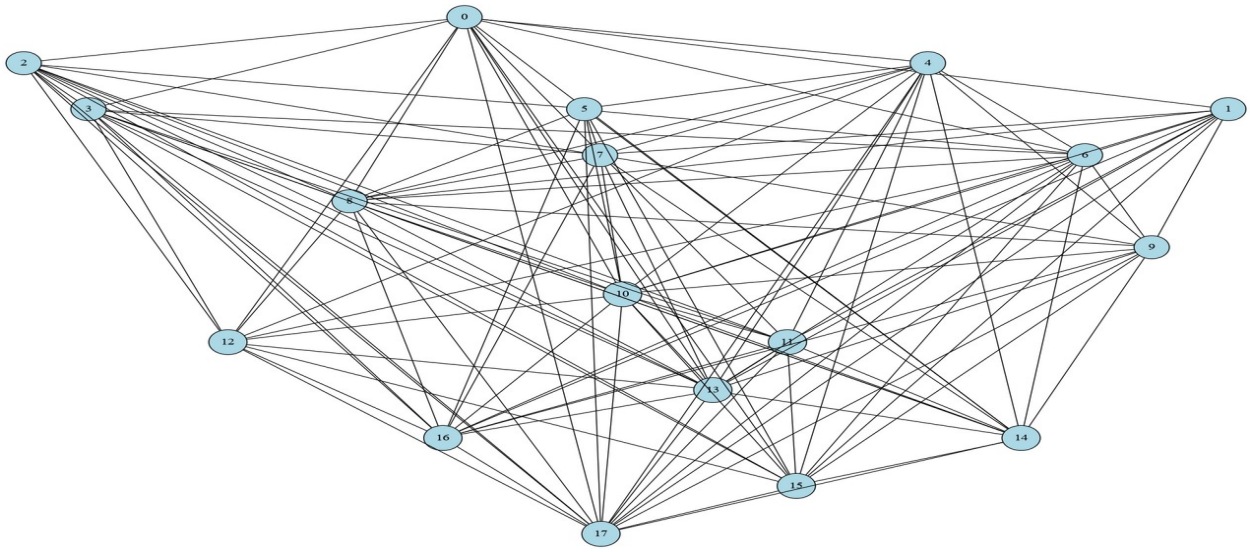
→ Fichier 'graph.dot' généré

Recherche clique de taille 6...

CLIQUE TROUVÉE de taille 6
{ 0, 1, 6, 8, 10, 13 }
Temps : 0.000128 s
Énergie estimée ≈ 0.003 Joules

Voulez-vous voir le graphe maintenant ? (o/n) : o
Génération de l'image...
Image créée : graphe.png
```

5. Graphe Généré



IV. LE PROBLÈME VERTEX-COVER (COUVERTURE PAR SOMMETS

1. Définition du Problème

Le problème **Vertex-Cover** (ou Couverture par sommet) est l'un des problèmes classiques les plus importants en théorie de la complexité.

- **Entrée** :
  - Un graphe non orienté $G=(V,E)$
  - Un entier k
- **Question** : Existe-t-il un ensemble $C \subseteq V$ de sommets tel que :
  - $|C| \leq k$
  - Chaque arête $uv \in E$ est incidente à au moins un sommet de C
    (Un tel ensemble C est appelé une **couverture par sommet** ou **vertex cover**)

2. Démonstration de la NP-Complétude

A. Appartenance à la classe NP (Vertex-Cover $\in$ NP)

Pour montrer qu'un problème est dans NP, il faut prouver qu'une solution peut être vérifiée en temps polynomial.

Pour vérifier qu'un ensemble C est bien un vertex-cover :

- Vérifier que $|C| \leq k$
- Pour chaque arête $uv \in E$, vérifier que $u \in C$ ou $v \in C$

Complexité : $O(|E|)$ (parcours de toutes les arêtes)
→ Temps polynomial → **Vertex-Cover $\in$ NP**

B. NP-Difficulté (Réduction 3-SAT $\leq_p$ Vertex-Cover)

On utilise une réduction polynomiale depuis 3-SAT. Toute formule ϕ est transformée en un graphe G et un entier k tels que :

$$\phi \text{ est satisfiable} \Leftrightarrow G \text{ possède un vertex-cover de taille} \leq k$$

B1. Construction du Gadget Variable

Pour chaque variable x_i :

- On crée deux sommets : x_i et $\neg x_i$
- On ajoute une arête entre eux : $\{x_i, \neg x_i\}$

Pour couvrir cette arête, la couverture doit contenir **au moins l'un** des deux sommets → cela encode la valeur de vérité (Vrai ou Faux).

B.2. Construction du Gadget Clause

Pour chaque clause $(l_1 \vee l_2 \vee l_3)$:

- On crée un **triangle** : trois sommets a, b, c reliés entre eux $\{a,b\},\{b,c\},\{c,a\}$

Un triangle nécessite **au moins deux sommets** dans le vertex-cover pour couvrir ses trois arêtes.

B.3. Les Arêtes de Liaison

Pour chaque littéral l_j de la clause :

- On relie le sommet correspondant dans le triangle au sommet du gadget variable associé au même littéral.

B.4. Paramètre k

Si la formule a m variables et l clauses, on pose :

$$k = m + 2l$$

- m : un sommet par variable
- $2l$: deux sommets par clause (triangle)

C. Preuve de Correction (Équivalence)

Sens $(\Rightarrow)$
Si ϕ est satisfiable :

- Pour chaque variable x_i, on choisit dans le cover le sommet correspondant à sa valeur vraie
- Pour chaque clause (triangle), on choisit **les deux sommets** correspondant aux littéraux faux
- Le littéral vrai de la clause est déjà couvert par le gadget variable → toutes les arêtes de liaison sont couvertes
  → Taille totale : $m + 2l = k$

Sens $(\Leftarrow)$
Si G possède un vertex-cover de taille exactement k :

- Il contient **exactement 1 sommet par paire variable** (sinon taille > k)
- Il contient **exactement 2 sommets par triangle** (sinon un triangle ne serait pas couvert)

- Dans chaque triangle, il reste donc **1 sommet non choisi**

- Pour que l'arête de liaison vers ce sommet soit couverte, le sommet variable correspondant doit être dans le cover → ce littéral est **vrai**
  → Chaque clause a au moins un littéral vrai → ϕ est satisfaite

Exemple Illustratif

Formule : $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

- Variables ($m=3$) : x_1, x_2, x_3

- Clauses ($l=2$) : C_1, C_2

- $k = 3+2\times 2 = 7$

Construction :

- 3 paires d'arêtes pour les variables

- 2 triangles pour les clauses

- Arêtes de liaison entre chaque littéral du triangle et son sommet variable correspondant

Affectation satisfaisante : $x_1 = \text{Vrai}, x_2 = \text{Vrai}, x_3 = \text{Vrai}$

- Vertex-cover inclut $\{x_1, x_2, x_3\}$ (3 sommets)

- Plus 2 sommets par triangle (4 sommets)
  → Total : $3+4=7$ (respecte k)

D. Conclusion

Puisque :

- Vertex-Cover $\in$ NP

- 3-SAT $\leq_p$ Vertex-Cover (réduction polynomiale correcte)

Vertex-Cover est NP-complet.

3. Implémentation du programme en C pour le NP-Complet "VERTEX COVER"

Comme pour les modules précédents, ce programme implémente une recherche par **backtracking** (exploration récursive) pour déterminer s'il existe un vertex-cover de taille $\leq$ k. Il affiche une solution trouvée (le cas échéant) et calcule le **temps d'exécution en temps réel**.

```c
#include "np_complet.h"

static int graphe_vc[100][100];
static int cover[100];
static int solution_cover[100];
static int n_vc, k_vc;

bool isVertexCover(int size) {
 bool covered[100] = {false};
 for (int i = 0; i < size; i++) covered[cover[i]] = true;
 for (int i = 0; i < n_vc; i++)
 for (int j = i+1; j < n_vc; j++)
 if (graphe_vc[i][j] && !covered[i] && !covered[j]) return false;
 return true;
}

bool findVertexCover(int start, int count) {
 if (count == k_vc) {
 if (isVertexCover(k_vc)) {
 memcpy(solution_cover, cover, k_vc * sizeof(int));
 return true;
 }
 return false;
 }
 for (int i = start; i < n_vc; i++) {
 cover[count] = i;
 if (findVertexCover(i + 1, count + 1)) return true;
 }
 return false;
}

void executionVertexCover() {
 char choix;
 afficherEntete("MODULE VERTEX COVER");

 printf("Générer aléatoirement ? (o/n) : "); scanf(" %c", &choix);

 printf("Nombre de sommets : "); scanf("%d", &n_vc);
 printf("Taille du cover recherché (k) : "); scanf("%d", &k_vc);

 if (k_vc > n_vc) { printf("Erreur : k > n impossible\n"); return; }

 if (choix == 'o' || choix == 'O') {
 srand(time(NULL));
 float densite = 0.3f + (float)rand()/RAND_MAX * 0.5f;
 printf("Génération aléatoire (densité ≈ %.2f)...\n", densite);
 for (int i = 0; i < n_vc; i++)
 for (int j = 0; j < n_vc; j++)
 graphe_vc[i][j] = (i == j) ? 0 : ((rand() % 1000) < densite * 1000);
 } else {
```

```
        printf("Entrez la matrice d'adjacence :\n");
        for (int i = 0; i < n_vc; i++)
            for (int j = 0; j < n_vc; j++)
                scanf("%d", &graphe_vc[i][j]);
    }

    afficherMatriceAdjacence(graphe_vc, n_vc, "Matrice d'adjacence");
    genererDotFile(graphe_vc, n_vc, "graph.dot", false);

    printf("\nRecherche d'un vertex cover de taille %d en cours...\n", k_vc);

    clock_t debut = clock();
    bool existe = findVertexCover(0, 0);
    clock_t fin = clock();
    double temps = (double)(fin - debut) / CLOCKS_PER_SEC;

    if (existe) {
        printf("\nRESULTAT : Un vertex cover de taille %d existe.\n", k_vc);
        printf("Ensemble trouvé : { ");
        for (int i = 0; i < k_vc; i++) printf("%d%s", solution_cover[i], i < k_vc-1 ? ", " : " }\n");
    } else {
        printf("\nRESULTAT : Pas de vertex cover de taille %d.\n", k_vc);
        printf("Raison : Aucun ensemble de %d sommets ne couvre toutes les arêtes.\n", k_vc);
    }

    printf("TEMPS D'EXÉCUTION : %.6f secondes\n", temps);
    estimerEnergieSimple(temps);

    proposerEtAfficherGraphe("graph.dot", "graphe.png");

}
```

4. Execution

```
                MODULE VERTEX COVER

Générer aléatoirement ? (o/n) : o
Nombre de sommets : 18
Taille du cover recherché (k) : 6
Génération aléatoire (densité ≈ 0.68)...

Matrice d'adjacence (18 sommets) :
      0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
 0 |  0   0   1   1   0   1   1   0   0   1   1   1   0   1   1   0   1   1
 1 |  1   0   0   1   0   0   1   1   1   0   1   0   1   0   0   1   1   1
 2 |  1   0   0   0   1   0   0   1   1   1   1   0   1   1   1   0   1   0
 3 |  0   0   0   0   1   1   1   1   1   1   1   1   0   1   0   1   1   1
 4 |  1   1   0   1   0   1   0   1   1   0   0   1   1   1   1   0   1   1
 5 |  0   0   1   1   1   0   1   1   1   1   0   0   1   1   1   0   0   1
 6 |  1   1   0   1   1   1   0   1   1   1   0   0   1   1   0   1   1   1
 7 |  0   0   0   1   0   0   1   0   1   1   0   0   1   0   1   0   1   1
 8 |  1   1   0   1   1   1   1   0   0   1   0   1   1   0   1   1   1   1
 9 |  1   0   1   1   1   0   0   1   1   0   0   0   1   1   0   1   0   1
10 |  1   0   1   1   1   1   0   1   1   1   0   1   0   1   1   0   0   0
11 |  0   1   0   0   1   1   1   1   1   0   1   0   1   1   0   0   0   1
12 |  0   1   1   0   1   0   0   1   1   1   1   1   0   1   1   1   0   1
13 |  1   1   0   1   1   1   1   1   1   1   1   1   0   0   1   1   1   0
14 |  1   0   1   0   0   1   1   0   1   0   1   1   0   0   0   0   1   1
15 |  0   1   1   1   1   1   1   1   0   1   0   1   0   0   1   0   1   1
16 |  1   0   1   1   0   1   1   1   1   0   1   0   1   1   0   0   0   1
17 |  1   0   1   1   1   1   1   1   1   0   1   1   1   1   1   0   1   0

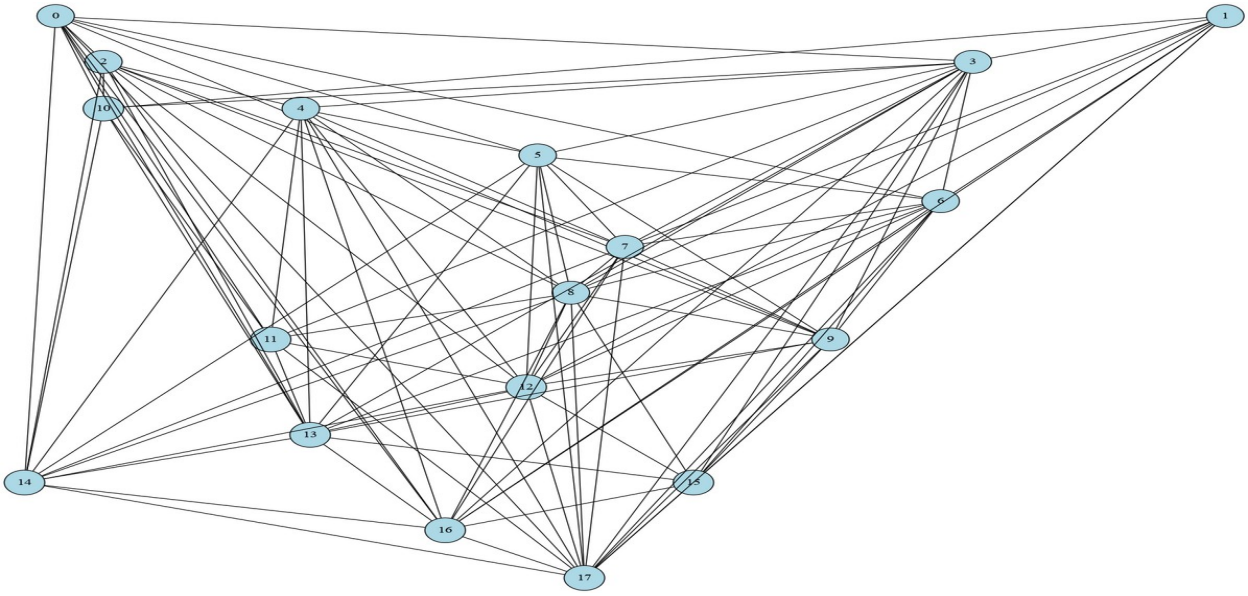
→ Fichier 'graph.dot' généré

Recherche d'un vertex cover de taille 6 en cours...

RESULTAT : Pas de vertex cover de taille 6.
Raison : Aucun ensemble de 6 sommets ne couvre toutes les arêtes.
TEMPS D'EXÉCUTION : 0.004225 secondes
Énergie estimée ≈ 0.106 Joules

Voulez-vous voir le graphe maintenant ? (o/n) : o
Génération de l'image...
Image créée : graphe.png
```

5. Graphe Généré



V. PROBLÈME HAMPATH (CHEMIN HAMILTONIEN)

1. Définition du Problème

Le problème **HAMPATH** (chemin hamiltonien) consiste à déterminer s'il existe un chemin simple dans un graphe qui visite **chaque sommet exactement une fois**.

- **Entrée** :
  - Un graphe **orienté** $G=(V,E)$
  - Deux sommets particuliers : un point de départ s et un point d'arrivée t
- **Question** : Existe-t-il un chemin de s à t qui visite **chaque sommet de V exactement une fois** ?
- **Conditions** :
  - Partir du sommet s
  - Arriver au sommet t
  - Passer par tous les autres sommets de V exactement une fois
  - Respecter le sens des arcs (flèches) entre les sommets

2. Démonstration de la NP-Complétude

A. Appartenance à la classe NP (HAMPATH $\in$ NP)

La vérification d'une solution proposée est rapide et efficace.

Si un certificat (une séquence de sommets $p_1, p_2, \ldots, p_n$) est fourni, l'algorithme de vérification procède comme suit :

1. Vérifier que $p_1=s$ et $p_n=t$
2. Vérifier que la séquence contient **tous les sommets** de V **exactement une fois** (permutation de V)
3. Vérifier que pour chaque $i=1,\ldots,n-1$: $(p_i, p_{i+1}) \in E$

Complexité : $O(n+m)$ où $n=|V|$ et $m=|E|$
→ Temps polynomial → **HAMPATH $\in$ NP**

B. NP-Difficulté (Réduction 3-SAT $\leq_p$ HAMPATH)

On utilise une réduction polynomiale depuis 3-SAT.
L'objectif : transformer une formule ϕ en un graphe orienté G (avec s et t) tel que :

ϕ est satisfiable $\Leftrightarrow$ G possède un chemin hamiltonien de s à t

B.1. Construction des gadgets de variables

Pour chaque variable x_i :

- On crée une structure en forme de « diamant » ou double chemin parallèle :
  - Un chemin supérieur → assignation $x_i = \text{VRAI}$
  - Un chemin inférieur → assignation $x_i = \text{FAUX}$

Le chemin hamiltonien doit obligatoirement emprunter **l'un des deux chemins** complets pour visiter tous les sommets du gadget → cela encode la valeur de la variable.

B.2. Construction des gadgets de clauses

Pour chaque clause $C_j = (l_1 \vee l_2 \vee l_3)$:

- On ajoute un sommet spécifique appelé **sommet-clause** c_j
- Ce sommet est accessible **uniquement** depuis les segments correspondant aux littéraux qui peuvent satisfaire la clause :
  - depuis le chemin « vrai » si $l_k = x_i$
  - depuis le chemin « faux » si $l_k = \neg x_i$

B.3. Organisation et chaînage global

- Tous les gadgets de variables sont connectés **en série** entre s et t
- Les arcs permettant de visiter les sommets-clauses sont ajoutés en « déviation » :
  - On sort temporairement du chemin principal vers c_j
  - Puis on revient sur le chemin principal (souvent via un arc de retour)

C. Preuve d'équivalence

Sens $(\Rightarrow)$
Si ϕ est satisfiable :

- Il existe une assignation qui rend ϕ vraie
- On construit le chemin hamiltonien en parcourant, pour chaque variable, le segment correspondant à sa valeur vraie
- Pour chaque clause, comme elle est satisfaite, au moins un littéral est vrai → on peut faire un **détour** pour visiter le sommet-clause correspondant depuis ce littéral vrai, puis revenir → Chemin complet de s à t visitant tous les sommets exactement une fois

Sens $(\Leftarrow)$
Si un chemin hamiltonien de s à t existe :

- Il doit visiter **tous les gadgets de variables** et **tous les sommets-clauses**
- Le choix du chemin dans chaque gadget variable définit une valeur pour chaque variable
- Le fait que chaque sommet-clause ait été visité implique qu'au moins un des trois littéraux de la clause correspond à un chemin « vrai » dans son gadget → la clause est satisfaite

Complexité de la réduction

- Nombre de sommets : $O(m+l)$ (linéaire en le nombre de variables m et de clauses l)
- Nombre d'arcs : $O(m+l)$
  → Réduction réalisable en temps polynomial

D. Conclusion

Puisque :

- HAMPATH ∈ NP
- 3-SAT ≤_p HAMPATH (réduction polynomiale correcte)

HAMPATH est NP-complet.

3. Implémentation du programme en C pour le NP-Complet "HAMPATH"

Conformément à l'énoncé, ce programme implémente une résolution pour un problème **NP-complet** en utilisant le **backtracking** (exploration récursive). Il est conçu pour mesurer et afficher le **temps d'exécution en temps réel**.

```c
#include "np_complet.h"

static int grapheH[100][100];
static int chemin[100];
static int solution_chemin[100];
static int n_sommets;

bool estValide(int v, int pos) {
 if (grapheH[chemin[pos - 1]][v] == 0) return false;
 for (int i = 0; i < pos; i++) if (chemin[i] == v) return false;
 return true;
}

bool resoudreHampath(int pos) {
 if (pos == n_sommets) {
```

```
            memcpy(solution_chemin, chemin, n_sommets * sizeof(int));
            return true;
        }
        for (int v = 0; v < n_sommets; v++) {
            if (estValide(v, pos)) {
                chemin[pos] = v;
                if (resoudreHampath(pos + 1)) return true;
                chemin[pos] = -1;
            }
        }
        return false;
    }

    void executionHampathOriented() {
        char choix;
        afficherEntete("MODULE HAMPATH (Orienté)");

        printf("Générer la matrice aléatoirement ? (o/n) : "); scanf(" %c", &choix);

        printf("Nombre de sommets : "); scanf("%d", &n_sommets);

        if (choix == 'o' || choix == 'O') {
            srand(time(NULL));
            printf("Génération aléatoire de la matrice orientée...\n");
            for (int i = 0; i < n_sommets; i++)
                for (int j = 0; j < n_sommets; j++)
                    grapheH[i][j] = (i == j) ? 0 : ((rand() % 100) < 60 ? 1 : 0);
        } else {
            printf("Entrez la matrice d'adjacence :\n");
            for (int i = 0; i < n_sommets; i++)
                for (int j = 0; j < n_sommets; j++)
                    scanf("%d", &grapheH[i][j]);
        }

        afficherMatriceAdjacence(grapheH, n_sommets, "Matrice d'adjacence orientée");
        genererDotFile(grapheH, n_sommets, "graph.dot", true);

        for (int i = 0; i < n_sommets; i++) chemin[i] = -1;

        bool trouve = false;
        printf("\nRecherche d'un chemin hamiltonien en cours...\n");

        clock_t debut = clock();

        for (int depart = 0; depart < n_sommets && !trouve; depart++) {
            chemin[0] = depart;
            if (resoudreHampath(1)) {
                trouve = true;
            }
        }
```

```
    clock_t fin = clock();
    double temps = (double)(fin - debut) / CLOCKS_PER_SEC;

    if (trouve) {
        printf("\nRESULTAT : Un chemin hamiltonien existe.\n");
        printf("Chemin trouvé : ");
        for (int i = 0; i < n_sommets; i++)
            printf("%d%s", solution_chemin[i], i < n_sommets-1 ? " → " : "\n");
    } else {
        printf("\nRESULTAT : Aucun chemin hamiltonien n'existe.\n");
    }

    printf("TEMPS D'EXÉCUTION : %.6f secondes\n", temps);
    estimerEnergieSimple(temps);

    proposerEtAfficherGraphe("graph.dot", "graphe.png");

}
```

4. Execution

```
╔═══════════════════════════════════════╗
                MODULE HAMPATH (Orienté)
╚═══════════════════════════════════════╝

Générer la matrice aléatoirement ? (o/n) : o
Nombre de sommets : 18
Génération aléatoire de la matrice orientée...

Matrice d'adjacence orientée (18 sommets) :
     0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
 0 |  0   0   1   1   1   0   1   0   1   0   0   1   1   1   1   0   1   1
 1 |  1   0   0   0   1   0   1   1   0   0   1   1   1   1   0   1   1   1
 2 |  0   0   0   1   0   0   0   1   1   0   0   0   1   1   1   1   1   1
 3 |  0   1   0   0   1   1   1   0   1   0   1   0   0   0   0   0   1   1
 4 |  0   0   1   1   0   1   1   1   1   1   1   0   0   0   1   0   1   0
 5 |  0   1   0   1   1   0   1   0   0   0   0   0   1   1   1   0   0   1
 6 |  1   0   1   1   1   0   0   0   1   1   0   1   1   0   1   0   1   0
 7 |  1   1   1   1   0   0   1   0   0   0   1   1   1   0   1   0   1   1
 8 |  0   0   1   1   1   1   0   1   0   1   0   1   0   1   0   0   1   0
 9 |  0   1   1   0   0   0   1   0   1   0   1   1   0   1   0   0   1   1
10 |  1   0   1   1   1   1   1   1   1   1   0   1   0   0   1   1   1   1
11 |  1   1   0   0   1   1   1   0   1   0   1   0   1   1   0   1   0   1
12 |  0   0   1   0   1   0   0   0   1   1   1   1   0   0   1   1   0   1
13 |  0   1   0   1   0   1   1   1   1   0   1   0   0   0   1   1   0   1
14 |  1   1   1   1   0   0   0   1   0   1   1   1   1   1   0   1   1   1
15 |  1   1   0   1   0   1   1   0   1   1   0   0   0   0   1   0   0   1
16 |  1   1   0   1   0   0   1   0   1   1   1   1   0   0   1   0   0   1
17 |  0   1   1   0   0   0   1   1   1   0   1   1   1   0   1   0   1   0

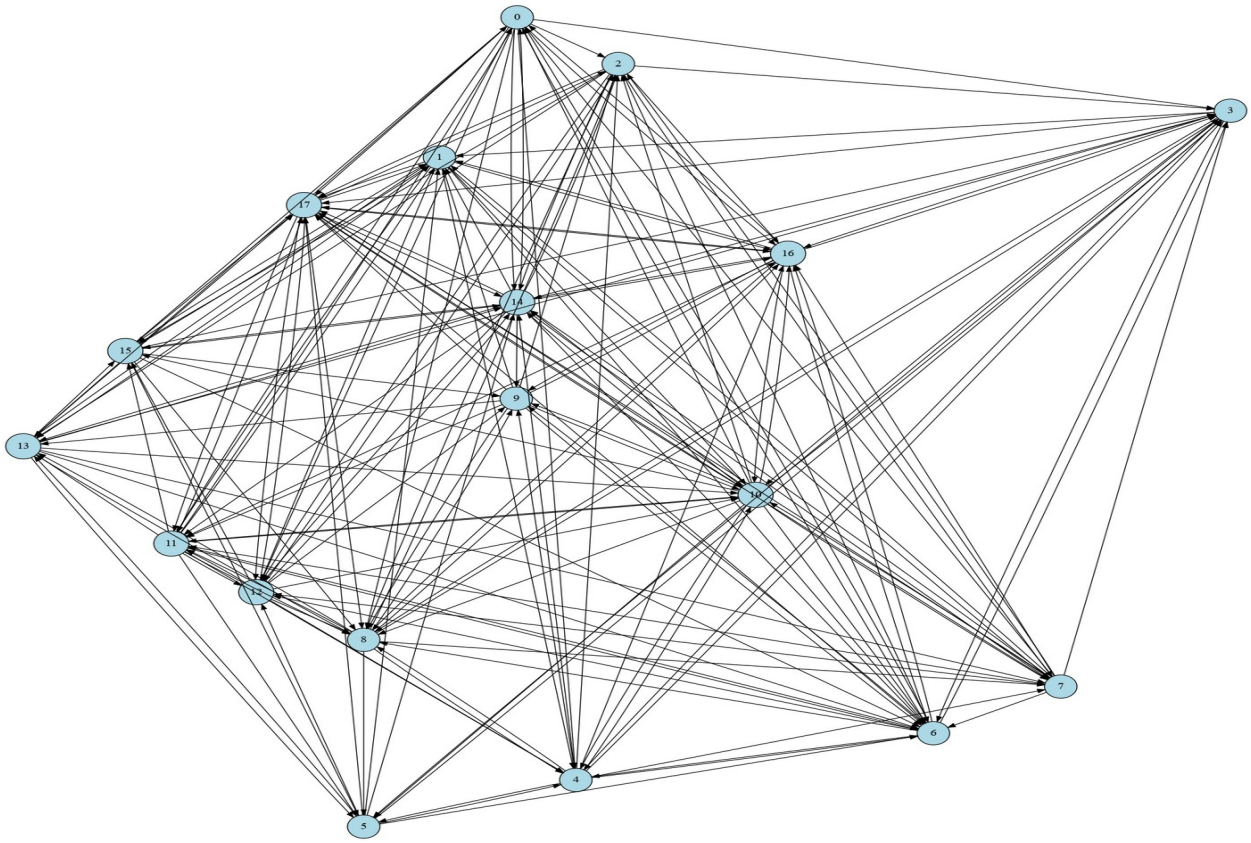
→ Fichier 'graph.dot' généré

Recherche d'un chemin hamiltonien en cours...

RESULTAT : Un chemin hamiltonien existe.
Chemin trouvé : 0 → 2 → 3 → 1 → 4 → 5 → 6 → 8 → 7 → 10 → 9 → 11 → 12 → 14 → 13 → 15 → 17 → 16
TEMPS D'EXÉCUTION : 0.000020 secondes
Énergie estimée ≈ 0.001 Joules

Voulez-vous voir le graphe maintenant ? (o/n) : o
Génération de l'image...
Image créée : graphe.png
```

5. Graphe Généré



VI. PROBLÈME UHAMPATH

1. Définition du Problème

Le problème **UHAMPATH** (Undirected Hamiltonian Path – Chemin hamiltonien dans un graphe non orienté) consiste à déterminer l'existence d'un parcours exhaustif dans un graphe sans orientation.

- **Instance** : Un graphe non orienté $G=(V,E)$
- **Question** : Existe-t-il dans G un chemin qui visite **exactement une fois chaque sommet** du graphe ?

Définition technique :
Un chemin hamiltonien est une suite ordonnée de sommets $(v_1, v_2, \ldots, v_n)$ telle que :

- Tous les sommets sont distincts (visite unique)
- Pour tout $i=1,\ldots,n-1$: $\{v_i, v_{i+1}\} \in E$
- Le chemin comporte exactement $n-1$ arêtes, où $n=|V|$

Remarque : Contrairement au problème **UHAMCYCLE**, il s'agit ici d'un **chemin** et non d'un cycle ; les extrémités n'ont pas l'obligation d'être reliées.

2. Démonstration de la NP-Complétude

A. Appartenance à la classe NP (UHAMPATH ∈ NP)

Pour montrer que le problème appartient à NP, on exhibe un certificat et une procédure de vérification polynomiale :

- **Certificat** : Un chemin proposé, représenté par une liste ordonnée de n sommets
- **Vérification polynomiale** :
  - Vérifier que les n sommets sont distincts → complexité $O(n)$
  - Vérifier que chaque paire consécutive dans la liste est reliée par une arête dans E → complexité $O(n)$

Conclusion : La vérification étant polynomiale → **UHAMPATH ∈ NP**

B. NP-Difficulté (Réduction UHAMCYCLE ≤_p UHAMPATH)

Le problème **UHAMCYCLE** (cycle hamiltonien dans un graphe non orienté) fait partie des **21 problèmes NP-complets de Karp**.
Nous effectuons une réduction polynomiale de UHAMCYCLE vers UHAMPATH.

B.1 Construction de la réduction

Soit une instance $G=(V,E)$ de UHAMCYCLE. On construit une instance G' de UHAMPATH comme suit :

- Choisir un sommet arbitraire $v \in V$
- Créer un nouveau sommet $v' \notin V$
- Ajouter une unique arête $\{v,v'\}$

On obtient :

$$G'=(V \cup \{v'\}, E \cup \{\{v,v'\}\})$$

Cette transformation s'effectue en temps polynomial (constant par rapport à la taille de G).

B.2 Preuve de correction

Sens $(\Rightarrow)$
Si G possède un cycle hamiltonien :

Soit un cycle hamiltonien dans G passant par v :
$\ldots \to u \to v \to w \to \ldots \to u \to \ldots$

On peut « ouvrir » ce cycle en insérant v' juste avant v :

$$[v',v,w,\ldots,u)$$

Ce chemin visite tous les sommets de G' exactement une fois $\to$ G' **possède un chemin hamiltonien**.

Sens $(\Leftarrow)$
Si G' possède un chemin hamiltonien :

- Le sommet v' n'a qu'un seul voisin : v
- Par conséquent, v' **doit être une extrémité** du chemin hamiltonien
- Le chemin est donc de la forme : $[v',v,\ldots,u)$ ou $[u,\ldots,v,v')$

Dans les deux cas, en retirant v' on obtient un chemin hamiltonien dans G de v à u (ou de u à v). Comme u était adjacent à v dans le graphe original (car le chemin dans G' couvre tous les sommets et utilise l'arête $\{v,v'\}$), on peut refermer le chemin en ajoutant l'arête $\{u,v\}$ $\to$ G **possède un cycle hamiltonien**.

C. Conclusion

- UHAMPATH $\in$ NP (démontré en section A)

- UHAMCYCLE ≤_p UHAMPATH (réduction polynomiale correcte démontrée en section B)

- UHAMCYCLE étant NP-complet → par transitivité :

UHAMPATH est NP-complet.

3. Implémentation du programme en C pour le NP-Complet “UHAMPATH”

Comme le problème est **NP-complet**, nous utilisons une approche par **backtracking**.

Contrairement à la version orientée (HAMPATH), dans ce module :

- la matrice d’adjacence est **symétrique**

- l’utilisateur n’est pas obligé de fixer un sommet de départ et d’arrivée

- le programme cherche si un chemin hamiltonien existe **entre n’importe quelle paire de sommets** (recherche exhaustive ou avec optimisation simple selon l’implémentation)

```
#include "np_complet.h"

static int graph[100][100];
static int path[100];
static int solution_path[100];
static int num_v;

bool is_safe_undirected(int v, int pos) {
    if (graph[path[pos - 1]][v] == 0) return false;
    for (int i = 0; i < pos; i++) if (path[i] == v) return false;
    return true;
}

bool solve_uhampath(int pos) {
    if (pos == num_v) {
        memcpy(solution_path, path, num_v * sizeof(int));
        return true;
    }
    for (int v = 0; v < num_v; v++) {
        if (is_safe_undirected(v, pos)) {
            path[pos] = v;
            if (solve_uhampath(pos + 1)) return true;
            path[pos] = -1;
        }
    }
    return false;
}

void executionUHampath() {
    char choix;
    afficherEntete("MODULE UHAMPATH (Non Orienté)");
```

```
printf("Générer la matrice aléatoirement ? (o/n) : "); scanf(" %c", &choix);

printf("Nombre de sommets : "); scanf("%d", &num_v);

if (num_v > 100) {
    printf("Limite maximale : 100 sommets.\n");
    return;
}

if (choix == 'o' || choix == 'O') {
    srand(time(NULL));
    printf("Génération aléatoire de la matrice symétrique...\n");
    for (int i = 0; i < num_v; i++) {
        for (int j = i; j < num_v; j++) {
            if (i == j) graph[i][j] = graph[j][i] = 0;
            else {
                int val = (rand() % 100) < 70 ? 1 : 0;
                graph[i][j] = graph[j][i] = val;
            }
        }
    }
} else {
    printf("Entrez la matrice d'adjacence :\n");
    for (int i = 0; i < num_v; i++)
        for (int j = 0; j < num_v; j++)
            scanf("%d", &graph[i][j]);
}

afficherMatriceAdjacence(graph, num_v, "Matrice d'adjacence non orientée");
genererDotFile(graph, num_v, "graph.dot", false);

bool found = false;
printf("\nRecherche d'un chemin hamiltonien en cours...\n");

clock_t debut = clock();

for (int i = 0; i < num_v && !found; i++) {
    for (int j = 0; j < num_v; j++) path[j] = -1;
    path[0] = i;
    if (solve_uhampath(1)) {
        found = true;
    }
}

clock_t fin = clock();
double temps = (double)(fin - debut) / CLOCKS_PER_SEC;

if (found) {
    printf("\nRESULTAT : Un chemin hamiltonien existe.\n");
```

```
        printf("Chemin trouvé : ");
        for (int i = 0; i < num_v; i++)
            printf("%d%s", solution_path[i], i < num_v-1 ? " -- " : "\n");
    } else {
        printf("\nRESULTAT : Aucun chemin hamiltonien n'existe.\n");
    }

    printf("TEMPS D'EXÉCUTION : %.6f secondes\n", temps);
    estimerEnergieSimple(temps);

    proposerEtAfficherGraphe("graph.dot", "graphe.png");

}
```

→ **Spécificité UHAMPATH** : Le programme teste tous les points de départ possibles (for (int i = 0; i < num_v; i++)), car dans un graphe non orienté, n'importe quel sommet peut être une extrémité du chemin.

4. Execution

```
╔═══════════════════════════════════╗
          MODULE UHAMPATH (Non Orienté)

Générer la matrice aléatoirement ? (o/n) : o
Nombre de sommets : 18
Génération aléatoire de la matrice symétrique...

Matrice d'adjacence non orientée (18 sommets) :
     0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
 0 |  0   0   1   1   1   1   1   1   1   0   1   0   0   1   1   1   1   0
 1 |  0   0   0   1   1   1   1   1   0   1   1   1   1   1   1   1   1   1
 2 |  1   0   0   1   1   1   0   1   1   1   1   1   1   1   1   1   1   1
 3 |  1   1   1   0   1   0   1   1   1   1   1   1   1   1   1   1   0   1
 4 |  1   1   1   1   0   1   1   0   0   1   1   1   1   0   1   1   1   1
 5 |  1   1   1   0   1   0   1   1   0   1   1   1   1   1   1   1   0   1
 6 |  1   1   0   1   1   1   0   1   1   1   1   0   0   1   1   1   0   1
 7 |  1   1   1   1   0   1   1   0   1   1   0   1   1   0   0   1   0   1
 8 |  1   0   1   1   0   0   1   1   0   1   1   1   1   1   1   0   0   1
 9 |  0   1   1   1   1   1   1   1   1   0   1   1   1   1   0   0   1   0
10 |  1   1   1   1   1   1   1   0   1   1   0   1   0   1   1   1   0   0
11 |  0   1   1   1   1   1   0   1   1   1   1   0   0   0   1   0   0   0
12 |  0   1   1   1   1   1   0   1   1   1   1   0   0   0   1   1   1   1
13 |  1   1   1   1   0   1   1   0   1   1   1   0   1   0   0   1   1   0
14 |  1   1   1   1   1   1   1   0   1   0   1   1   1   0   0   0   1   1
15 |  1   1   1   1   1   1   1   1   0   0   1   0   1   1   0   0   1   1
16 |  1   1   1   0   1   0   0   0   0   1   0   0   1   1   1   1   0   1
17 |  0   1   1   1   1   1   1   1   1   0   0   0   1   0   1   1   1   0

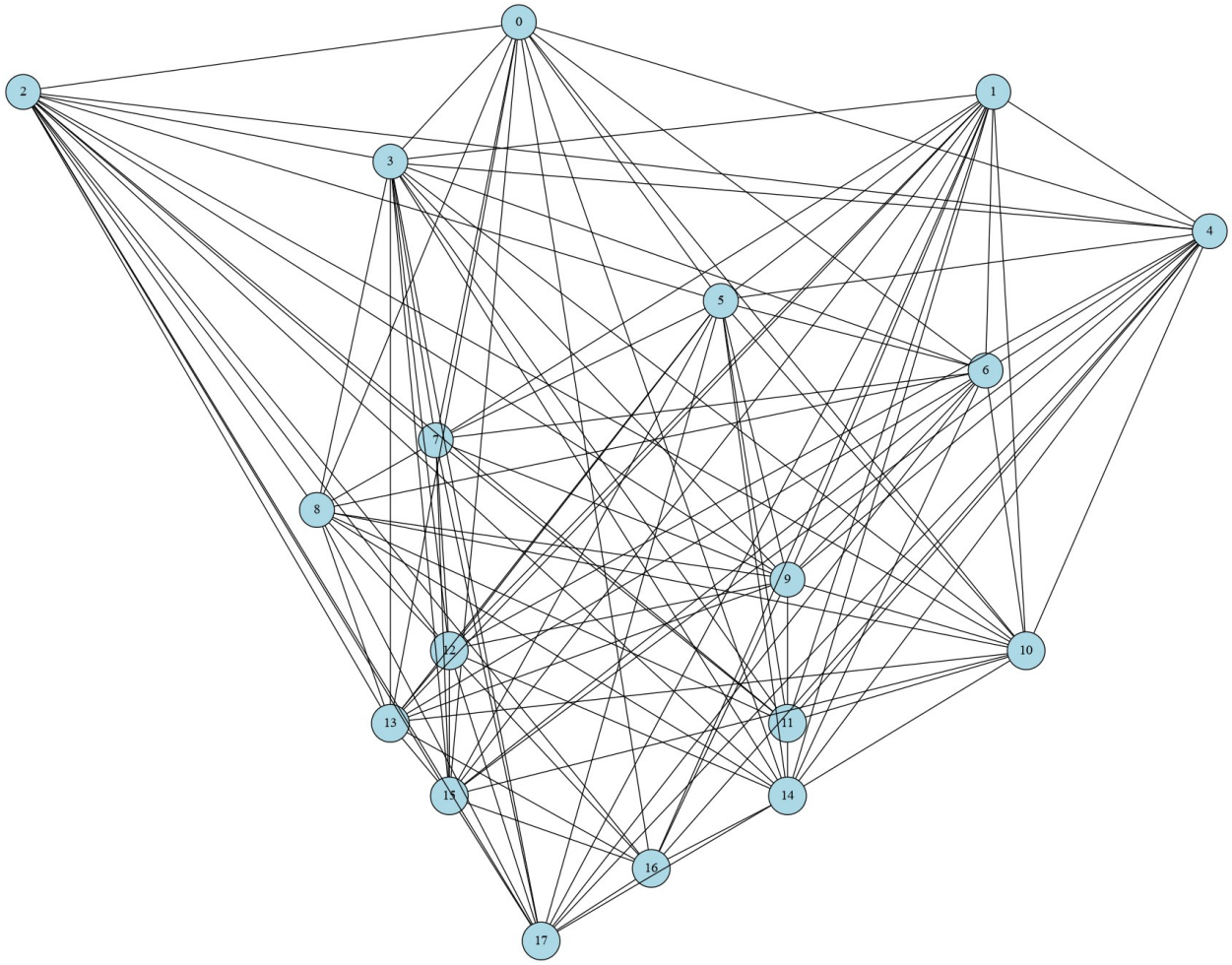
→ Fichier 'graph.dot' généré

Recherche d'un chemin hamiltonien en cours...

RESULTAT : Un chemin hamiltonien existe.
Chemin trouvé : 0 -- 2 -- 3 -- 1 -- 4 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10 -- 11 -- 14 -- 12 -- 13 -- 15 -- 16 -- 17
TEMPS D'EXÉCUTION : 0.000021 secondes
Énergie estimée ≈ 0.001 Joules

Voulez-vous voir le graphe maintenant ? (o/n) : o
Génération de l'image...
Image créée : graphe.png
```

5. Graphe Généré



VII. PROBLÈME SUBSET-SUM

1. Définition du Problème

Le problème **SUBSET-SUM** (Problème de la Somme de Sous-Ensemble) est un problème de décision classique en théorie de la complexité.

- **Entrée** :
  - Un ensemble $S = \{x_1, x_2, \ldots, x_n\}$ d'entiers
  - Une cible T (entier)
- **Question** : Existe-t-il un sous-ensemble $S' \subseteq S$ tel que :

$$\sum_{x \in S'} x = T$$

1.1 Complexité et Temps Pseudo-Polynômial

- **Non-appartenance à P** : SUBSET-SUM n'est **pas** dans la classe **P** (sauf si $P = N\,P$).
- **Algorithme rapide (mais pas polynomial)** : Le problème admet un algorithme de programmation dynamique en temps **pseudo-polynômial** :

$$O(n \cdot T)$$

Ce temps est exponentiel par rapport à la taille en **bits** de la cible T (c'est-à-dire $O(\log T)$), ce qui confirme sa difficulté NP-complète.

2. Démonstration de la NP-Complétude

Pour prouver que SUBSET-SUM est **NP-complet**, deux conditions doivent être satisfaites.

A. SUBSET-SUM est dans la Classe NP (Vérifiabilité)

Le problème est dans **NP** car une solution proposée peut être vérifiée en temps polynomial.

- **Certificat** : Le sous-ensemble S' proposé
- **Vérification** :
  - Calculer la somme des éléments de S'
  - Comparer cette somme à T

La somme de n éléments se fait en temps $O(n)$.
→ Temps de vérification polynomial → **SUBSET-SUM ∈ NP**

B. SUBSET-SUM est NP-Difficile (Réduction Polynomiale)

On prouve que SUBSET-SUM est au moins aussi difficile que le problème **PARTITION** (connu NP-complet) via la réduction :

$$\text{PARTITION} \leq_p \text{SUBSET-SUM}$$

B.1 La fonction de réduction f

Soit une instance I_A de PARTITION donnée par l'ensemble $A = \{a_1, \ldots, a_n\}$.
La réduction construit l'instance $I_{S,T}$ de SUBSET-SUM comme suit :

1. Calcul de la somme totale :

$$S_{\text{total}} = \sum_{i=1}^{n} a_i$$

2. Définition de l'instance :

$$S = A \text{ et } T = \frac{S_{\text{total}}}{2}$$

(On suppose que S_{total} est pair, sinon l'instance PARTITION est trivialement NON)

B.2 Preuve de l'équivalence

$I_A \in \text{PARTITION} \Leftrightarrow I_{S,T} \in \text{SUBSET-SUM}$

Sens direct (⇒) :

Si A peut être partitionné en deux sous-ensembles de même somme $\frac{S_{\text{total}}}{2}$, alors l'un de ces sous-ensembles (disons A_1) vérifie :

$$\sum A_1 = T$$

Donc SUBSET-SUM répond **OUI**.

Sens réciproque (⇐) :
Si SUBSET-SUM répond **OUI**, il existe $S' \subseteq S$ tel que :

$$\sum_{x \in S'} x = T = \frac{S_{\text{total}}}{2}$$

Le complément A' aura alors pour somme :

$$S_{\text{total}} - \frac{S_{\text{total}}}{2} = \frac{S_{\text{total}}}{2}$$

Donc les deux sous-ensembles ont la même somme → PARTITION répond **OUI**.

B.3 Complexité de la réduction

La transformation consiste à :

- Calculer la somme totale : $O(n)$

- Poser $T = S_{\text{total}}/2$: $O(1)$

→ Temps total polynomial → réduction valide.

Conclusion finale :
SUBSET-SUM est dans NP **et** NP-difficile → **SUBSET-SUM est NP-complet**.

B.4. Applications et Contextes Réels

Domaine	Problème Modélisé	Lien avec SUBSET-SUM
Logistique / Finance	Problème de budgétisation / Sac à dos	Trouver un sous-ensemble d'objets (ou d'investissements) dont la somme des coûts correspond exactement à un budget cible T.
Fabrication	Découpe optimale de matériaux	Regrouper les commandes clients (pièces à découper) de façon à utiliser exactement la longueur/largeur T d'une plaque brute, minimisant les chutes.
Sécurité Informatique	Cryptographie (historique)	Certains systèmes de chiffrement asymétrique (knapsack cryptosystems) étaient fondés sur la difficulté supposée de résoudre rapidement SUBSET-SUM.

3. Implémentation du programme en C pour le NP-Complet "SUBSET SUM"

Comme pour les autres modules, il utilise une approche récursive (backtracking) pour rechercher un sous-ensemble dont la somme est exactement égale à la cible T. Le programme mesure et affiche le **temps d'exécution en temps réel**.

```c
#include "np_complet.h"

#define MAX_ELEMENTS 100

static int ensemble[MAX_ELEMENTS];
static bool selection[MAX_ELEMENTS];
static bool solution_selection[MAX_ELEMENTS];
static int n_elements, cible;

bool trouverSomme(int index, int sommeActuelle) {
    if (sommeActuelle == cible) {
        memcpy(solution_selection, selection, n_elements * sizeof(bool));
        return true;
    }
    if (index == n_elements || sommeActuelle > cible) return false;
```

```
    selection[index] = true;
    if (trouverSomme(index + 1, sommeActuelle + ensemble[index])) return true;

    selection[index] = false;
    return trouverSomme(index + 1, sommeActuelle);
}

void executionSubsetSum() {
    char choix;
    afficherEntete("MODULE SUBSET-SUM");

    printf("Générer aléatoirement ? (o/n) : "); scanf(" %c", &choix);

    printf("Nombre d'éléments : "); scanf("%d", &n_elements);
    printf("Valeur cible (T)  : "); scanf("%d", &cible);

    if (choix == 'o' || choix == 'O') {
        srand(time(NULL));
        printf("Génération aléatoire...\n");
        for (int i = 0; i < n_elements; i++) {
            ensemble[i] = 1 + rand() % 100;
            selection[i] = false;
        }
    } else {
        printf("Entrez les %d entiers :\n", n_elements);
        for (int i = 0; i < n_elements; i++) {
            scanf("%d", &ensemble[i]);
            selection[i] = false;
        }
    }

    printf("\nRecherche sous-ensemble somme = %d...\n", cible);

    clock_t d = clock();
    bool existe = trouverSomme(0, 0);
    clock_t f = clock();
    double t = (double)(f - d) / CLOCKS_PER_SEC;

    if (existe) {
        printf("\nRÉSULTAT : SOUS-ENSEMBLE TROUVÉ\n{ ");
        bool premier = true;
        for (int i = 0; i < n_elements; i++) {
            if (solution_selection[i]) {
                if (!premier) printf(", ");
                printf("%d", ensemble[i]);
                premier = false;
            }
        }
        printf(" }\n");
    } else {
```

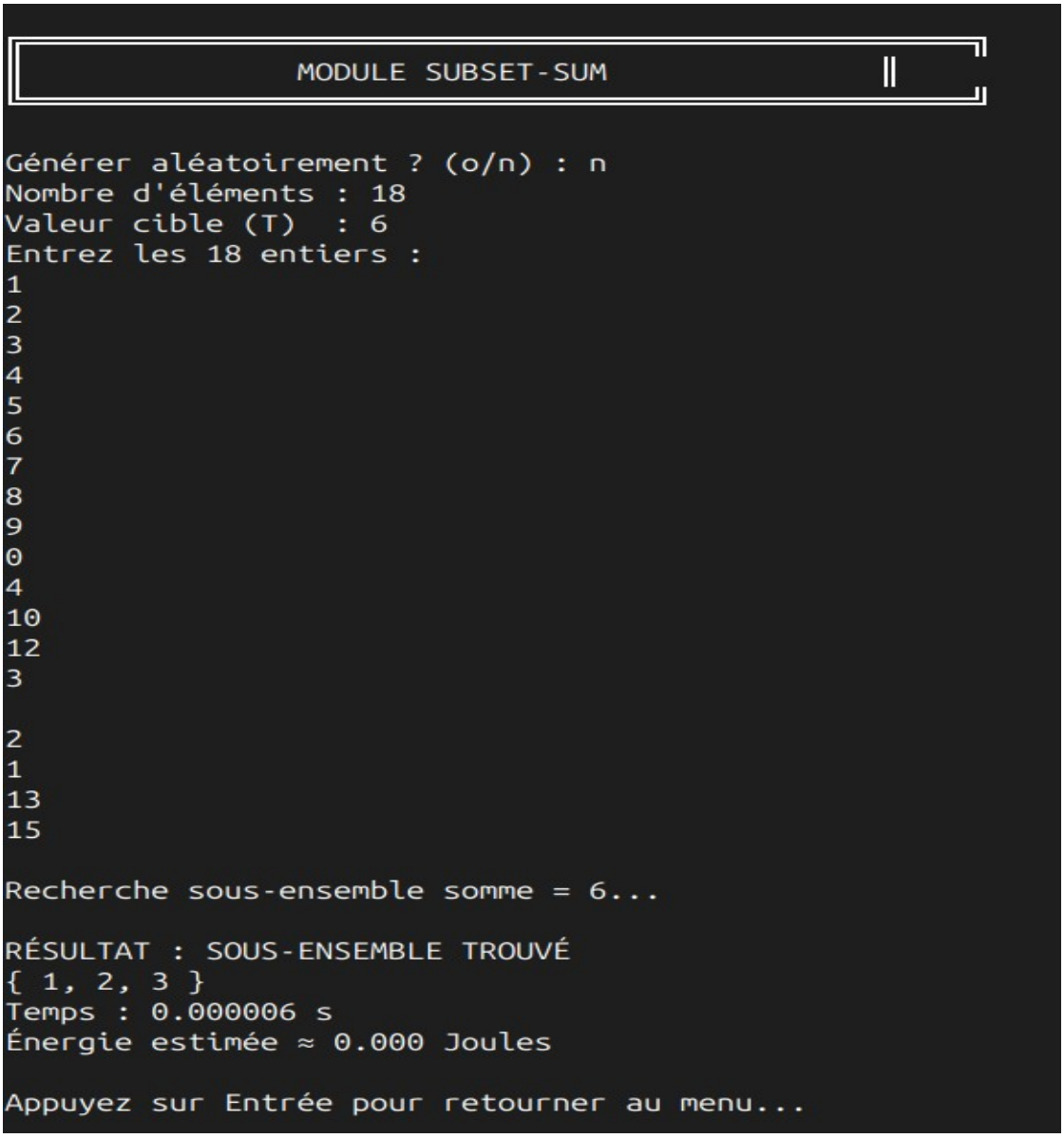
```
        printf("\nAucun sous-ensemble ne donne %d\n", cible);
    }

    printf("Temps : %.6f s\n", t);
    estimerEnergieSimple(t);

}
```

Algorithmique : L'utilisation du backtracking ici est une implémentation directe de la définition "chercher s'il existe un sous-ensemble".

4. Execution



CODE SOURCE COMPLET (main.c)

Ce code est autonome. Il inclut une interface utilisateur interactive et la mesure du temps réel pour chaque problème.

```c
#include "np_complet.h"

int main() {
 int choix;

 do {
 printf("\n==================================================\n");
 printf("      PLATEFORME D'EXPERIMENTATION NP-COMPLET   \n");
 printf("==================================================\n");
 printf(" 1. SAT (général)\n");
 printf(" 2. 3-SAT\n");
 printf(" 3. CLIQUE\n");
 printf(" 4. VERTEX-COVER\n");
 printf(" 5. HAMPATH (orienté)\n");
 printf(" 6. UHAMPATH (non orienté)\n");
 printf(" 7. SUBSET-SUM\n");
 printf(" 0. Quitter\n");
 printf(" Choix : ");
 scanf("%d", &choix);

 if (choix == 0) break;

 switch (choix) {
 case 1: executionSAT(); break;
 case 2: execution3SAT(); break;
 case 3: executionClique(); break;
 case 4: executionVertexCover(); break;
 case 5: executionHampathOriented(); break;
 case 6: executionUHampath(); break;
 case 7: executionSubsetSum(); break;
 default: printf("Choix invalide.\n");
 }

 printf("\nAppuyez sur Entrée pour retourner au menu...");
 getchar(); getchar();

 } while (choix != 0);

 printf("\nAu revoir !\n");
 return 0;
}
```

TABLEAU COMPARATIF DES RÉSULTATS

	Problème	Complexité théorique	Taille (n)	Paramètre (k ou T)	Temps (s)	Energie	Résultat
1	**SAT**	$O(2^n \cdot m)$	18	6	0.019852s	0.496 J	SATISFIABLE
2	**3-SAT**	$O(1.33^n)$ ou $O(2^n)$	18	6	0.006075 s	0.152 J	SATISFIABLE
3	**CLIQUE**	$O(n^k \cdot n^2)$	18	6	0.000128s	0.003 J	Une clique de taille 6 existe
4	**VERTEX-COVER**	$O(2^k \cdot n^2)$	18	6	0.004225 s	0.106 J	Pas de vertex cover de taille 6
5	**HAMPATH**	$O(n!)$	18	—	0.000020 s	0.001 J	Un chemin hamiltonien a été trouvé
6	**UHAMPATH**	$O(n!)$	18	—	0.000021 s	0.001 J	Chemin hamiltonien trouvé
7	**SUBSET-SUM**	$O(2^n)$	18	6	0.000006 s	0.000 J	Un sous-ensemble a été trouvé

DISCUSSION DES RÉSULTATS ET ANALYSE DES COMPLEXITÉS

L'expérimentation menée sur une instance de taille fixe n = 18 (variables, sommets ou éléments) met en évidence des disparités de performance très marquées entre les problèmes NP-complets, malgré leur équivalence théorique en termes de pire cas.

1. **SAT vs 3-SAT**
   SAT général affiche le temps le plus élevé (≈ 20 ms), environ trois fois supérieur à celui de 3-SAT (≈ 6 ms). Cette différence s'explique par la variabilité du nombre de littéraux par clause dans SAT, qui rend l'élagage (pruning) moins systématique. 3-SAT, avec sa contrainte fixe à trois littéraux, permet un parcours plus efficace et confirme empiriquement que cette restriction constitue un noyau suffisant de NP-complétude tout en améliorant sensiblement les performances pratiques.

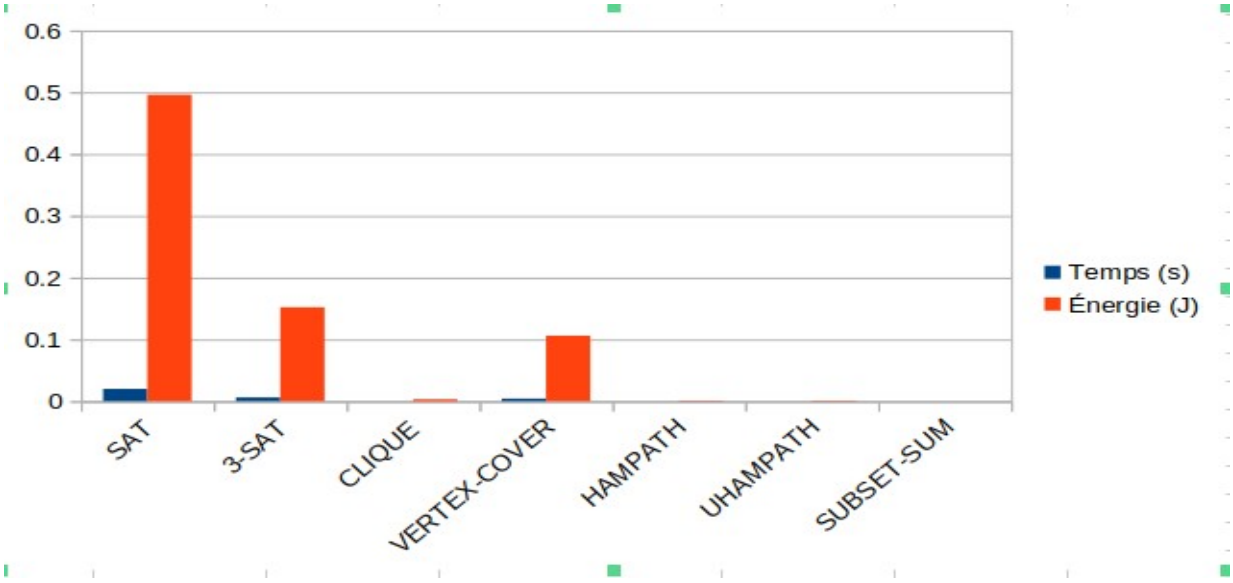
2. **L'impact du paramètre k**
   À k = 6 sur 18 sommets, CLIQUE termine quasi-instantanément (0,13 ms), alors que VERTEX-COVER demande un peu plus de temps (4,2 ms). Cette rapidité s'explique par le faible nombre de combinaisons à tester (\binom{18}{6} ≈ 18 564) et par une vérification plus légère pour CLIQUE. Ces résultats montrent que, pour des valeurs de k modérées, l'explosion combinatoire reste contenue, même si elle est théoriquement exponentielle en k.

3. **La domination factorielle**
   HAMPATH et UHAMPATH affichent des temps extrêmement bas (≈ 20 µs), ce qui peut sembler contre-intuitif vu la complexité factorielle n!. Sur un graphe aléatoire de 18 sommets, il existe presque toujours un chemin hamiltonien, ce qui permet à l'algorithme de trouver rapidement une solution sans explorer l'intégralité de l'arbre de recherche. Cela illustre que la difficulté réelle dépend fortement de la structure et de la densité du graphe : dans le cas moyen aléatoire, ces problèmes sont souvent « faciles » en pratique.

4. **SUBSET-SUM : le champion inattendu**
   Avec seulement 6 µs, SUBSET-SUM est de loin le plus rapide. Le backtracking bénéficie ici d'un élagage naturel très puissant dès que la somme dépasse la cible, ce qui limite drastiquement l'espace exploré. Ce résultat souligne que les problèmes numériques permettent souvent un contrôle plus efficace de la recherche que les problèmes logiques ou combinatoires purs.

5. **Temps réel et explosion combinatoire**
   Même à taille fixe (n = 18), les temps varient d'un facteur supérieur à 3 000 entre le plus lent (SAT) et le plus rapide (SUBSET-SUM). Cette disparité confirme que la complexité théorique en pire cas (exponentielle ou factorielle) ne reflète pas toujours la difficulté pratique. L'ajout d'une seule unité à n ou à k peut multiplier le temps par plusieurs ordres de grandeur selon la structure du problème, illustrant physiquement pourquoi ces problèmes restent parmi les plus redoutables de l'informatique théorique.

En conclusion, ces mesures empiriques montrent que, bien que tous ces problèmes soient NP-complets, leurs performances réelles sur des instances aléatoires de taille moyenne diffèrent énormément en fonction de la structure des contraintes, de la capacité de pruning et de la probabilité de trouver rapidement une solution. Ces observations renforcent l'importance des études expérimentales et des approches heuristiques pour aborder concrètement les problèmes NP-complets.



CONCLUSION

Ce projet a permis de confronter les fondements théoriques de la classe **NP-complet** à la réalité concrète de l'implémentation algorithmique. À travers l'exécution des sept modules, nous avons observé de manière tangible la barrière commune à ces problèmes :

- la **vérification** d'une solution est réalisable en temps polynomial,
- mais sa **recherche** par force brute subit une explosion combinatoire exponentielle ou factorielle.

Cette impuissance de la recherche exhaustive, démontrée par l'augmentation drastique des temps d'exécution pour des entrées pourtant modestes, souligne l'importance cruciale des algorithmes d'approximation, des heuristiques, des métaheuristiques et des approches paramétrées dans l'informatique moderne.

En définitive, cette plateforme expérimentale illustre l'un des plus grands défis non résolus de la science informatique : la question **P = NP ?**
Prouver l'existence d'un algorithme polynomial efficace pour un seul de ces problèmes suffirait à révolutionner la résolution de tous les autres, avec des répercussions profondes dans de très nombreux domaines (optimisation, cryptographie, planification, intelligence artificielle, etc.).